

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

—o0o—

BÁO CÁO DỰ ÁN

RICE MONITORING SYSTEM

Hệ thống giám sát và phân tích dữ liệu lúa thời gian thực

Sinh viên thực hiện: Võ Hữu Nghĩa
Mã số sinh viên: 3124410231
Chuyên ngành: Khoa học Máy tính

TP. HỒ CHÍ MINH, THÁNG 04/2026

Mục lục

I	KIẾN THỨC NỀN TẢNG	2
I.1	ẢNH VỆ TINH LÀ GÌ? (Raster Data)	2
I.1.1	Raster vs Vector	2
I.1.2	Cấu trúc dữ liệu của một ảnh Raster trong bộ nhớ	2
I.1.3	Sentinel-2 — Band nào dùng để tính NDVI?	2
I.1.4	Định dạng file	3
I.2	THÔNG TIN ĐỊA LÝ ĐÍNH KÈM (Geospatial Metadata)	3
I.2.1	CRS — Hệ tọa độ tham chiếu (Coordinate Reference System)	3
I.2.2	Affine Transform Matrix — Ma trận biến đổi	3
I.3	THƯ VIỆN (<code>rasterio</code>) — CẦU NỐI PYTHON ↔ FILE ẢNH	4
I.3.1	Rasterio là gì?	4
I.3.2	Khái niệm cốt lõi: Dataset Object	4
I.4	QUẢN LÝ BỘ NHỚ CƠ BẢN	4
I.4.1	Tính toán RAM tiêu thụ	4
II	CẤU TRÚC ĐẦY ĐỦ CỦA MA TRẬN AFFINE	5
II.1	Tại sao e âm?	5
II.2	Truy cập từng phần tử của Transform trong rasterio	5
II.3	Pixel size (GSD) là gì?	6
II.4	Bounding Box là gì?	6
II.4.1	Trường hợp lý tưởng ($b, d = 0$ - Ảnh không bị xoay)	6
II.4.2	Trường hợp tổng quát ($b, d \neq 0$ - Ảnh bị xoay nghiêng)	7
II.5	Ứng dụng nhân ma trận Affine	7
II.6	Tọa độ trả về: Góc pixel hay Tâm pixel?	8
II.7	Tổng kết Logic Xử lý & Checklist	8

Chương I

KIẾN THỨC NỀN TẢNG

I.1 ẢNH VỆ TINH LÀ GÌ? (Raster Data)

I.1.1 Raster vs Vector

Trong GIS (Hệ thống thông tin địa lý) có 2 kiểu dữ liệu chính:

- **Vector:** Lưu theo dạng điểm, đường, vùng (polygon). Ví dụ: ranh giới tỉnh thành.
- **Raster:** Lưu dưới dạng **lưới ô vuông (Grid)**, mỗi ô là một pixel có giá trị số. → Ảnh vệ tinh là Raster.

I.1.2 Cấu trúc dữ liệu của một ảnh Raster trong bộ nhớ

Hãy hình dung ảnh Raster là một **mảng 3 chiều (3D Array)**:

```
[Band(dải), Row(hàng), Column(cột)] <- NumPy shape
4          10980          10980 <- kích thước thực Sentinel-2
```

- **Band (Băng tần):** Mỗi band là 1 "kênh" ánh sáng khác nhau (Đỏ, Xanh, Hồng ngoại...).
- **Row × Column:** Chiều cao × Chiều rộng ảnh tính bằng số pixel.
- **Giá trị mỗi pixel:** Là số nguyên **uint16** (0 → 65535), biểu diễn cường độ phản xạ ánh sáng.

I.1.3 Sentinel-2 — Band nào dùng để tính NDVI?

Band	Tên	Bước sóng	Độ phân giải	Dùng để
B04	RED	665 nm	10m/px	Ánh sáng đỏ cây hấp thụ
B08	NIR	842 nm	10m/px	Hồng ngoại cây phản xạ

→ Chỉ cần 2 band: B04 và B08 để tính NDVI.

I.1.4 Định dạng file

- **GeoTIFF** (.tif): Định dạng phổ biến nhất, chứa cả dữ liệu pixel lẫn thông tin địa lý (CRS, Transform).
 - **JPEG2000** (.jp2): Định dạng nén Sentinel-2 gốc tải về từ Copernicus Hub.
-

I.2 THÔNG TIN ĐỊA LÝ ĐÍNH KÈM (Geospatial Metadata)

Điều khác biệt lớn nhất của ảnh vệ tinh so với ảnh chụp thông thường là nó có **tọa độ thực địa** gắn liền. Khi mở file, bạn không chỉ đọc pixel mà còn cần đọc 2 thứ quan trọng:

I.2.1 CRS — Hệ tọa độ tham chiếu (Coordinate Reference System)

- Mô tả cách "**chiều**" bề mặt cầu Trái Đất lên mặt phẳng 2D.
- Sentinel-2 thường dùng **EPSG:32648** (UTM Zone 48N) cho khu vực Việt Nam.
- Được biểu diễn bằng chuỗi **WKT** hoặc mã **EPSG**.

Ví dụ: "EPSG:32648" → UTM Zone 48N → Phù hợp Đồng bằng sông Cửu Long.

I.2.2 Affine Transform Matrix — Ma trận biến đổi

Đây là ma trận 3x3 ánh xạ từ tọa độ pixel (col, row) sang **tọa độ phẳng (Planar Coordinates - X, Y)**. Để chuyển tiếp từ không gian phẳng này sang **tọa độ địa lý mặt cầu (Kinh độ, Vĩ độ)**, dữ liệu cần được đưa qua các phương trình biến đổi của hệ quy chiếu (CRS).

$$\begin{pmatrix} x_{\text{real}} \\ y_{\text{real}} \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} col \\ row \\ 1 \end{pmatrix}$$

Trong đó:

- **(c, f)**: Tọa độ thực địa của góc trên-trái của ảnh (origin).
- **a**: Kích thước pixel theo chiều X (thường = +10.0 mét với Sentinel-2 10m).
- **e**: Kích thước pixel theo chiều Y (thường = -10.0 vì trục Y đảo ngược: hàng 0 ở trên).
- **b, d**: Thường = 0 nếu ảnh không bị xoay.

Tại sao cần matrix này? Để sau khi tính NDVI xong, có thể ghi kết quả ra file mới mà vẫn giữ nguyên tọa độ thực địa.

I.3 THƯ VIỆN (rasterio) — CẦU NỐI PYTHON ↔ FILE ẢNH

I.3.1 Rasterio là gì?

rasterio là thư viện Python wrapper của **GDAL** (Geospatial Data Abstraction Library - viết bằng C++). Nó giúp Python đọc/ghi các định dạng GeoTIFF, JP2, HDF5... mà không cần tự parse binary file.

I.3.2 Khái niệm cốt lõi: Dataset Object

Khi mở file với **rasterio**, nhận về một **Dataset Object**.

```
dataset = rasterio.open("data/01_raw/T48PXS_B04.jp2")

# Metadata
dataset.count          # Band number (usually = 1 for jp2)
dataset.width          # Column number (pixel)
dataset.height         # Row number (pixel)
dataset.dtypes         # Data type ('uint16',)
dataset.crs            # Coordinate Reference System
dataset.transform      # Affine Transform Matrix

data = dataset.read(1) # shape: (height, width)

dataset.close()
```

I.4 QUẢN LÝ BỘ NHỚ CƠ BẢN

I.4.1 Tính toán RAM tiêu thụ

```
1 pixel Sentinel-2 = uint16 = 2 bytes
1 band B04         = 10980 * 10980 * 2 bytes
                  = 241,120,800 bytes
                  ~ 230 MB (cho 1 band)

Cả 4 band (B02,B03,B04,B08) = ~920 MB
```

Chương II

CẤU TRÚC ĐẦY ĐỦ CỦA MA TRẬN AFFINE

$$\begin{pmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{pmatrix}$$

Ký hiệu	Tên gọi	Giá trị trong ảnh test	Ý nghĩa
c	X-origin	560000.0	Tọa độ X của góc trên-trái
f	Y-origin	1075120.0	Tọa độ Y của góc trên-trái
a	X pixel size	+10.0	Mỗi bước sang phải 1 cột = thêm 10 mét
e	Y pixel size	-10.0	Mỗi bước xuống 1 hàng = giảm 10 mét
b, d	Rotation	0.0	Ảnh không bị xoay

II.1 Tại sao e âm?

Đây là điểm gây nhầm lẫn nhiều nhất với người mới:

Hệ tọa độ MẢNG (NumPy):

```
Row 0 <- Hàng ĐẦU TIÊN
Row 1
Row 2
...
Row N <- Hàng CUỐI CÙNG
```

Hệ tọa độ BẢN ĐỒ (thực địa):

Y tăng lên = về phía Bắc (Lên)
Y giảm xuống = về phía Nam (Xuống)

Row 0 trong mảng = Vị trí **phía Bắc nhất** trên bản đồ (Y lớn nhất).
Row 1 = bước xuống 1 hàng → đi về phía Nam → Y **giảm**.
→ $e = -10.0$ vì "đi xuống 1 hàng pixel = giảm 10 mét tọa độ Y".

II.2 Truy cập từng phần tử của Transform trong rasterio

```
t = dataset.transform # object affine.Affine

t.a # pixel size theo X (thường = +10.0)
t.b # rotation x (thường = 0.0)
t.c # X tọa độ góc trên-trái
t.d # rotation y (thường = 0.0)
```

```
t.e    # pixel size theo Y (thường = -10.0)
t.f    # Y tọa độ góc trên-trái
```

II.3 Pixel size (GSD) là gì?

Pixel size (hay Ground Sampling Distance — GSD) là **kích thước thực địa của 1 pixel** trên mặt đất.

Sentinel-2 Band 4 và Band 8: mỗi pixel = 10m × 10m
(1 pixel đại diện cho 10m × 10m diện tích thực)

Độc pixel size từ đâu?

Pixel size nằm ngay trong ma trận Affine:

- **Chiều X** → lấy giá trị tuyệt đối của `transform.a` → $|t.a| = 10.0$
- **Chiều Y** → lấy giá trị tuyệt đối của `transform.e` → $|t.e| = 10.0$

Dùng `abs()` vì `e` âm nhưng kích thước pixel luôn là khoảng cách vật lý (số dương).

II.4 Bounding Box là gì?

Bounding Box (hộp giới hạn) là hình chữ nhật **nhỏ nhất** bao quanh toàn bộ ảnh, được mô tả bằng 4 tọa độ cực đại thực địa:

```

      <- left <-                                -> right ->
      |                                           |
TOP (North)  +-----+-----+-----+-----+ ^ top
      |                                           |
              TOÀN BỘ ẢNH
      |                                           |
BOTTOM(South) +-----+-----+-----+-----+ v bottom
```

II.4.1 Trường hợp lý tưởng ($b, d = 0$ - Ảnh không bị xoay)

Hệ phương trình tọa độ:

$$\begin{aligned}x &= a \cdot col + c \\y &= e \cdot row + f\end{aligned}$$

Để tìm giới hạn, ta xét các biên của ma trận ảnh:

- Để x_{min} thì $col = 0 \rightarrow left = c$
- Để x_{max} thì $col = width \rightarrow right = c + a \cdot width$
- Để y_{max} thì $row = 0 \rightarrow top = f$

- Để y_{min} thì $row = height \rightarrow bottom = f + e \cdot height$

Ví dụ minh họa:

$$\begin{aligned} left &= 560000.0 \\ right &= 560000.0 + 10.0 \times 512 = 565120.0 \\ top &= 1075120.0 \\ bottom &= 1075120.0 + (-10.0) \times 512 = 1070000.0 \end{aligned}$$

II.4.2 Trường hợp tổng quát ($b, d \neq 0$ - Ảnh bị xoay nghiêng)

Hệ phương trình tọa độ đầy đủ:

$$\begin{aligned} x &= a \cdot col + b \cdot row + c \\ y &= d \cdot col + e \cdot row + f \end{aligned}$$

Xác định tọa độ (x, y) tại 4 góc của ma trận ảnh:

- Góc trên-trái ($col = 0, row = 0$): $x_1 = c, y_1 = f$
- Góc trên-phải ($col = width, row = 0$): $x_2 = a \cdot width + c, y_2 = d \cdot width + f$
- Góc dưới-trái ($col = 0, row = height$): $x_3 = b \cdot height + c, y_3 = e \cdot height + f$
- Góc dưới-phải ($col = width, row = height$): $x_4 = a \cdot width + b \cdot height + c, y_4 = d \cdot width + e \cdot height + f$

Tọa độ hộp giới hạn (Bounding Box) chứa toàn bộ ảnh xoay:

$$\begin{aligned} left &= \min(x_1, x_2, x_3, x_4) \\ right &= \max(x_1, x_2, x_3, x_4) \\ top &= \max(y_1, y_2, y_3, y_4) \\ bottom &= \min(y_1, y_2, y_3, y_4) \end{aligned}$$

II.5 Ứng dụng nhân ma trận Affine

Đây là ứng dụng trực tiếp của phép nhân ma trận. Phương trình tổng quát chuyển đổi tọa độ điểm ảnh sang tọa độ thực địa:

$$\begin{pmatrix} x_{\text{real}} \\ y_{\text{real}} \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} col \\ row \\ 1 \end{pmatrix}$$

Khai triển hệ phương trình (khi ảnh không bị xoay, tức là $b = 0, d = 0$):

$$\begin{aligned} x_{\text{real}} &= a \cdot col + c \\ y_{\text{real}} &= e \cdot row + f \end{aligned}$$

II.6 Tọa độ trả về: Góc pixel hay Tâm pixel?

Trong quá trình xử lý hệ thống thông tin địa lý (GIS), cần phân biệt rõ 2 quy ước tính toán tọa độ điểm ảnh:

- **Góc trên-trái của pixel:** Rasterio mặc định sử dụng chuẩn này. Ví dụ với góc tọa độ '(row=0, col=0)' $\rightarrow$ '(560000, 1075120)'.
- **Tâm pixel (Center Pixel):** Tọa độ bị dịch chuyển (offset) thêm một nửa độ lớn của pixel size theo mỗi trục. Ví dụ '(row=0, col=0)' $\rightarrow$ '(560005, 1075115)'.

Lưu ý triển khai: Trong dự án thực tế của báo cáo này, hệ thống đã được can thiệp để implement logic lấy **tâm pixel** nhằm tăng độ chính xác khi đối chiếu với dữ liệu GPS (tọa độ GPS trên thực địa mặc định được coi là tâm của một vùng không gian).

II.7 Tổng kết Logic Xử lý & Checklist

Checklist xác thực kiến thức

- ✓ Hiểu rõ `transform.a` = pixel size theo trục X và `transform.e` = pixel size theo trục Y (thường là số âm).
- ✓ Nắm vững lý do `e` mang giá trị âm: Trong hệ tọa độ mảng, chỉ số hàng (row) tăng dần từ trên xuống dưới, tương ứng với việc tọa độ Y trên thực địa đang giảm dần (đi về phía Nam).
- ✓ Làm chủ phương trình ánh xạ tọa độ: $x = a \cdot col + c$ và $y = e \cdot row + f$.
- ✓ Tự tính toán được Bounding Box (khung bao giới hạn) từ tập hợp tham số `c`, `f`, `a`, `e`, `width`, `height`.
- ✓ Nắm được hành vi mặc định của `pixel_to_coords(0,0)` là trả về tọa độ ở góc trên-trái (top-left).

Bảng ánh xạ hàm xử lý

Hàm xử lý	Nguồn dữ liệu đầu vào	Công thức / Logic tính toán
<code>get_pixel_size()</code>	<code>transform.a</code> , <code>transform.e</code>	Lấy trị tuyệt đối: $(\text{abs}(a), \text{abs}(e))$
<code>get_bounds()</code>	<code>transform.c/f/a/e</code> & W/H	Biên phải: $right = c + a \cdot W$ Biên dưới: $bottom = f + e \cdot H$
<code>pixel_to_coords()</code>	<code>transform.a/c/e/f</code>	$x = a \cdot col + c$ $y = e \cdot row + f$ (*Cộng thêm $\frac{a}{2}, \frac{e}{2}$ nếu dùng tâm pixel)